



SDU International Summer School

Introduction to R

Day 6 - Data Visualization

Paola Vazquez-Castillo

10 August 2026

Contents

1	Introduction	2
1.1	Why we visualize?	2
1.2	The data	2
2	Base R	3
2.1	Scatter plot: <code>plot()</code>	3
2.1.1	Making the plot readable: titles, labels, color	5
2.2	Lines plot: <code>plot()</code>	6
2.3	Adding a reference line	7
2.4	Distributions <code>hist()</code> and <code>boxplot()</code>	8
2.5	Counts across categories: <code>barplot()</code>	11
3	The log lesson: when an axis spans orders of magnitude	12
4	The tidyverse plots: <code>ggplot2</code>	14
4.1	<code>geom_point()</code>	15
4.2	<code>geom_line()</code>	15
4.3	Transparency	16
4.4	A third variable	17
4.5	Adding a trend line: <code>geom_smooth()</code>	18
4.6	Distributions in <code>ggplot</code> : histogram and boxplot	19
4.7	Labels, titles, and themes	21
4.8	Scales and the log axis	22
4.9	Putting it all together: several variables in one plot	23
4.10	Multiple panels with <code>facet_wrap()</code>	24
5	Missing data: package <code>nanianr</code>	25
6	Combining plots: <code>patchwork()</code>	28

7	What viz to choose?	31
8	Glossary	32
9	Exercises	33

1 Introduction

We have seen that R is a free software environment for statistical computing, but one of the most attractive features of R is their data visualization tools. In R you are able to customize that data that you would like to have in your graphic. The basic idea is that you set up a plotting area, and then you add elements to it.

1.1 Why we visualize?

Numbers in a table tell you what a value is. A plot tells you what a variable is doing, its shape, its spread, and how it relates to other variables. We visualize for two different reasons, and it helps to know which one you are in:

- 1) Exploratory plots: these are quick and rough plots made for one self. The idea is not for them to be pretty but to be informative.
- 2) Communication plots: these are polished and labelled plots made for someone else (a reader, an examiner).

For case 1) base R plots are sufficient. For case 2) `ggplot()` part of the `tidyverse` is preferred.

1.2 The data

At the end of Day 5 we end by joining the student sleep records to the study-level information. This is the data we will use for today's class. We load the two files (telling R to treat blank cells as missing), then bring in the university and semester for each student with a `left_join`.

```
library(tidyverse)

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.1      v readr      2.2.0
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2    4.0.3      v tibble    3.3.1
## v lubridate  1.9.5      v tidyr     1.3.2
## v purrr      1.2.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to b

# Attach each study's university and semester to every student row
students <- left_join(sleep, studies, by = "study")

head(students)

##   student_id study gender first_generation underrepresented avg_sleep_hours
## 1         293    1 Female                0                0              6.17
## 2         294    1 Female                0                0              6.78
```

```
## 3      297      1 Female      0      1      4.23
## 4      303      1 Female      0      0      6.86
## 5      304      1 Female      0      1      6.80
## 6      305      1  Male      1      1      5.78
##  daytime_sleep_minutes sleep_midpoint_minutes bedtime_variability
## 1              11.7              500.3              0.0500
## 2              49.7              359.5              0.0476
## 3             178.5              398.2              0.1639
## 4              26.1              532.1              0.7854
## 5              71.6              354.0              0.1538
## 6              78.1              541.3              1.0689
##  nights_tracked_fraction prior_gpa term_gpa term_units term_load_z
## 1              0.931      4.00      3.76      49      0.112
## 2              0.862      4.00      3.55      49      0.112
## 3              0.586      3.00      3.59      46     -0.604
## 4              0.931      3.80      3.46      39     -2.275
## 5              0.931      3.17      3.50      39     -2.275
## 6              0.724      2.37      2.54      49      0.112
##              university semester cohort_code cohort_n
## 1 Carnegie Mellon University Spring 2018      lac2      77
## 2 Carnegie Mellon University Spring 2018      lac2      77
## 3 Carnegie Mellon University Spring 2018      lac2      77
## 4 Carnegie Mellon University Spring 2018      lac2      77
## 5 Carnegie Mellon University Spring 2018      lac2      77
## 6 Carnegie Mellon University Spring 2018      lac2      77
```

2 Base R

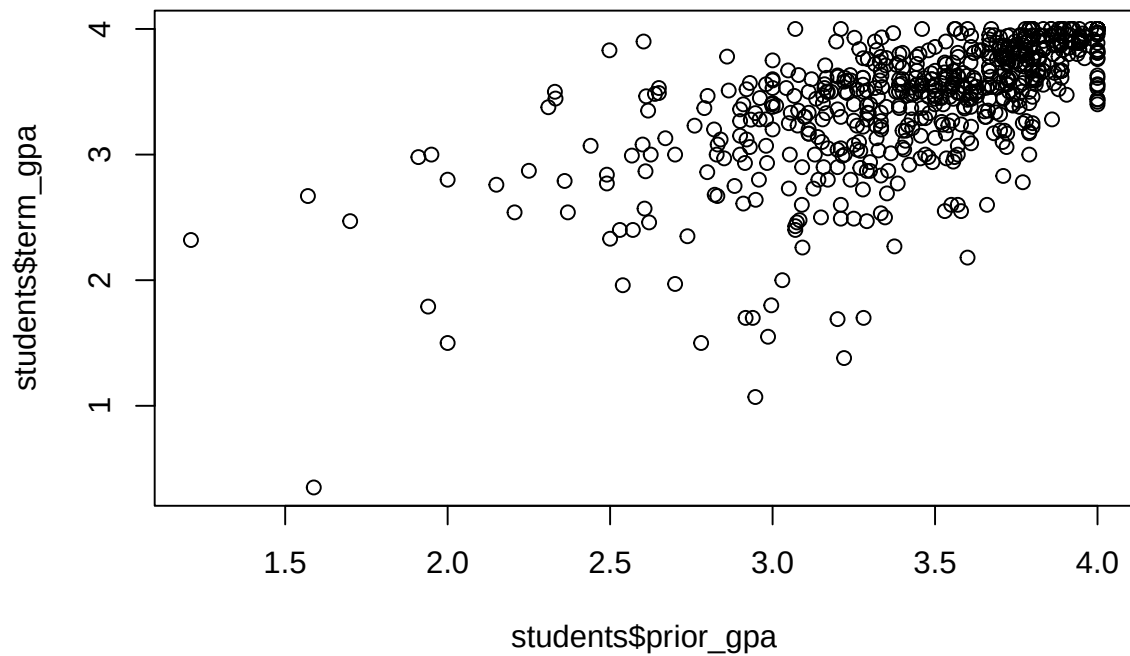
Base R plots are the simplest form of visualization and recommended when we are doing a preliminary analysis.

2.1 Scatter plot: `plot()`

As we learned on the first day, our most basic command for a graph is `plot()`. Its first two arguments are by default the x and the y coordinates. There must always be one element of x per one element of y. That means that x and y must have the same length. If you only provide one vector of values, this is used as the y coordinate, and it is plotted against the index values (“position number”) of the vector.

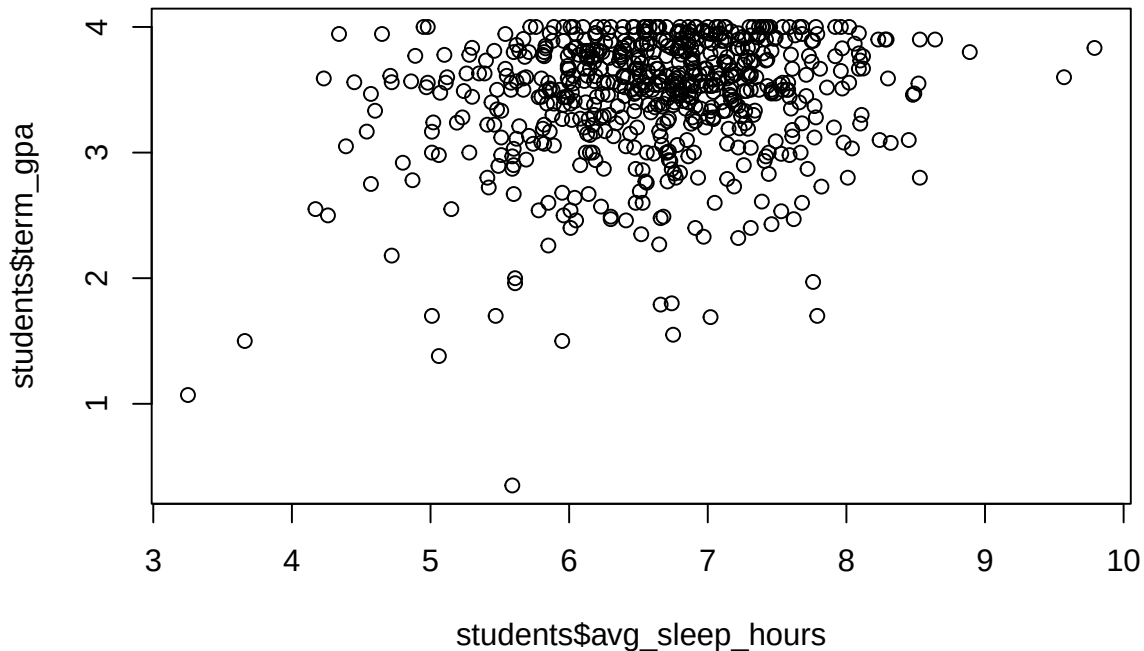
The default `plot()` shows points. This type of plot is often referred as a scatter plot and this is the correct type of plot when you want to **explore** the relationship between two numeric variables. To see what a clear relationship looks like, start with a student’s prior GPA against their current one.

```
plot(students$prior_gpa, students$term_gpa)
```



The points fall along a rising diagonal: students who did well before tend to do well again. That upward band is what a real relationship looks like on a scatter. Now a more interesting question: do students who sleep more earn a higher GPA?

```
plot(students$avg_sleep_hours, students$term_gpa)
```



This one is far messier. GPA piles up near the top across almost the whole sleep range, with only the handful of very-low-sleep students clearly lower. Two things are worth naming here, because both are real:

- 1) The cluster in the middle is a solid mass of overlapping points, so we cannot tell whether a few or a great many students sit there. This is called overplotting.
- 2) GPA cannot exceed 4.0, so the points flatten against the top of the plot. This is a ceiling effect, and it dampens any relationship at the high end.

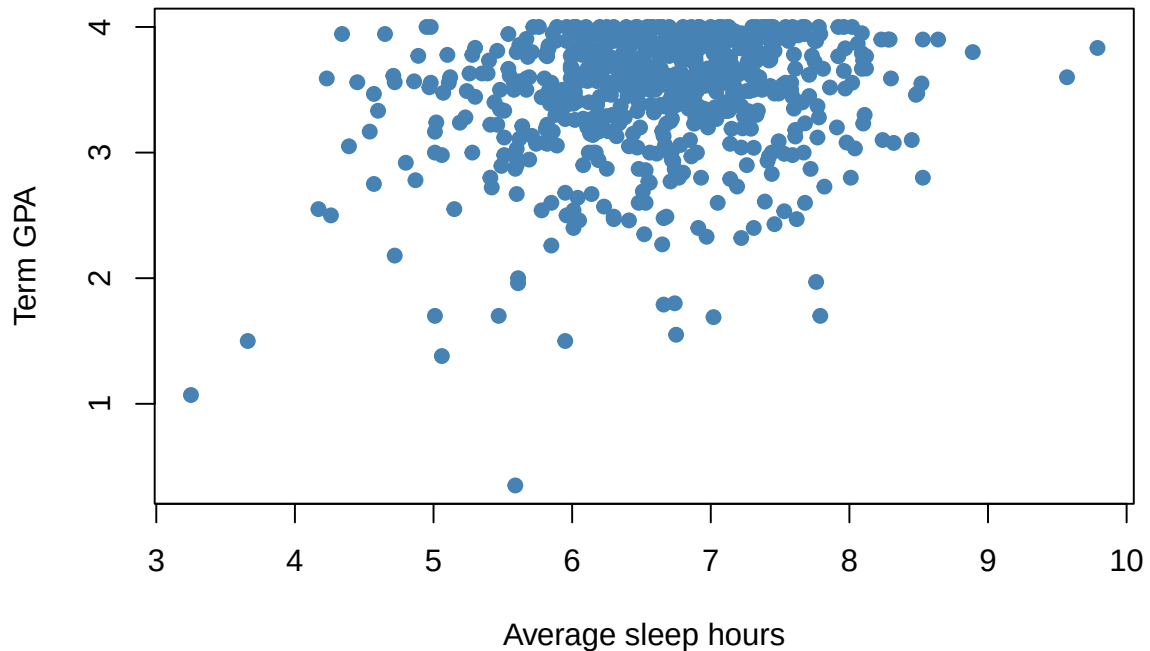
Real behavioural data often looks like this, weak and messy rather than a tidy line, and reading it honestly is a skill in itself. Both problems have fixes.

2.1.1 Making the plot readable: titles, labels, color

The arguments `main`, `xlab`, and `ylab` add a title and axis labels, `col` sets colour, and `pch` sets the point shape.

```
plot(students$avg_sleep_hours, students$term_gpa,
     main = "Sleep and term GPA",
     xlab = "Average sleep hours",
     ylab = "Term GPA",
     col = "steelblue",
     pch = 19)
```

Sleep and term GPA



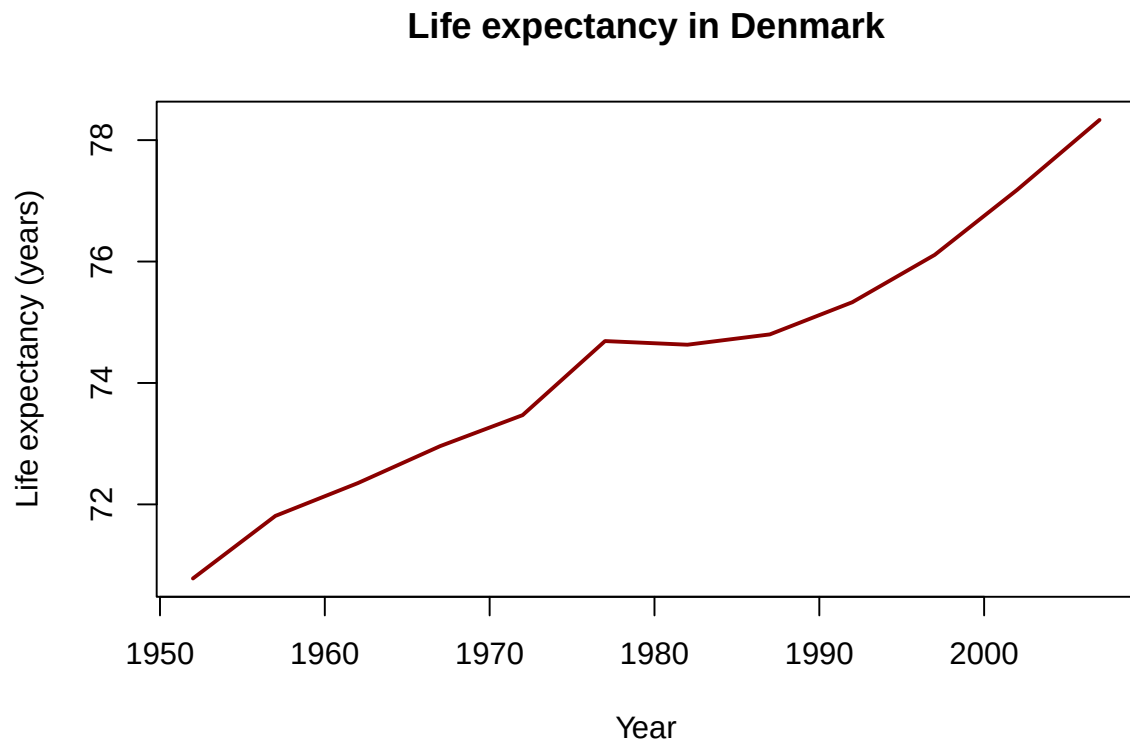
2.2 Lines plot: plot()

When the x axis is time, a line shows progression. The sleep data is not a time series, so we will use the data from the gapminder package on life expectancy, GDP per capita, and population for 142 countries between 1952 and 2007, in increments of 5 years.

```
# install.packages("gapminder")
library(gapminder)

denmark <- gapminder |> filter(country == "Denmark")

plot(denmark$year, denmark$lifeExp,
     type = "l",
     main = "Life expectancy in Denmark",
     xlab = "Year",
     ylab = "Life expectancy (years)",
     col = "darkred", lwd = 2)
```



The rule of thumb: a scatter for a relationship, a line for a trend over time.

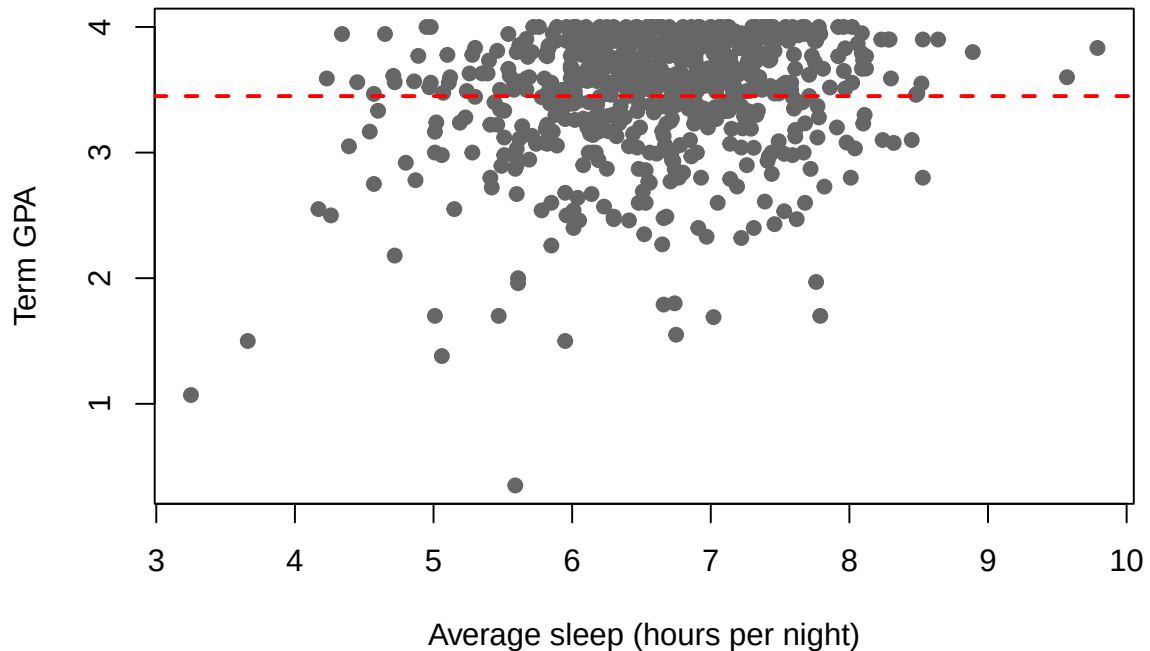
2.3 Adding a reference line

Once a plot exists, you can add to it. `abline()` draws a straight line: horizontal with `h =`, vertical with `v =`, or any line via intercept and slope with `a =` and `b =`.

```
plot(students$avg_sleep_hours, students$term_gpa,
     main = "With a reference line at the mean GPA",
     xlab = "Average sleep (hours per night)",
     ylab = "Term GPA",
     pch = 19, col = "grey40")

abline(h = mean(students$term_gpa, na.rm = TRUE),
       col = "red", lwd = 2, lty = 2)
```

With a reference line at the mean GPA



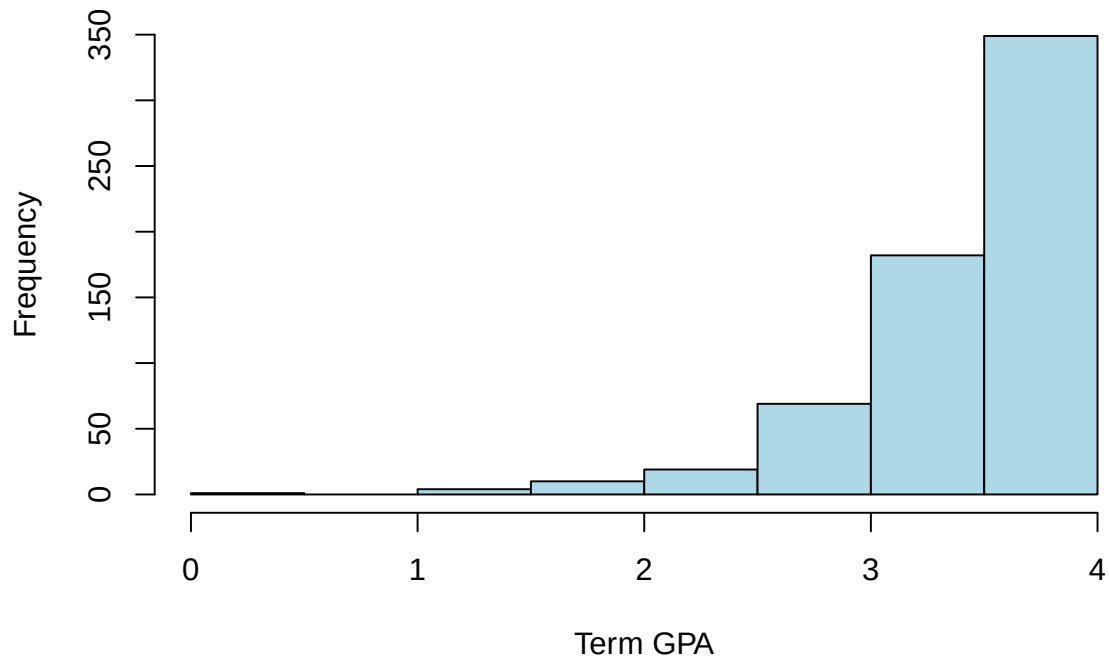
2.4 Distributions `hist()` and `boxplot()`

A histogram is a useful tool to graphically represent the distribution of a quantitative (numerical) variable. It shows us the *shape* and *spread* of our data by dividing it into bins (intervals) and counting how many observations fall into each bin. It shows the overall distribution of our data distribution and the concentration of data points. It is bar plot. The bar height represents the frequency (count) of observations in that interval, while the bar width represents the size of the interval (usually all bars have equal width).

A histogram shows the shape of one numeric variable. We do a histogram with the function `hist()`.

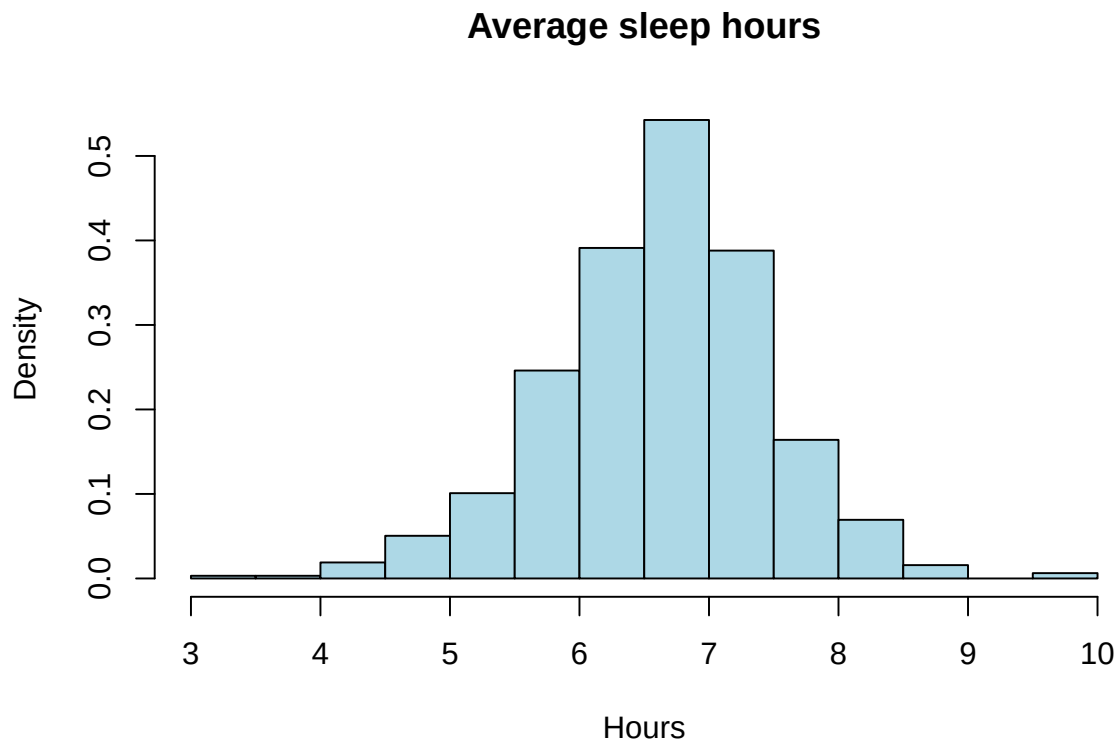
```
hist(students$term_gpa,  
     main = "Distribution of term GPA",  
     xlab = "Term GPA",  
     col  = "lightblue")
```

Distribution of term GPA



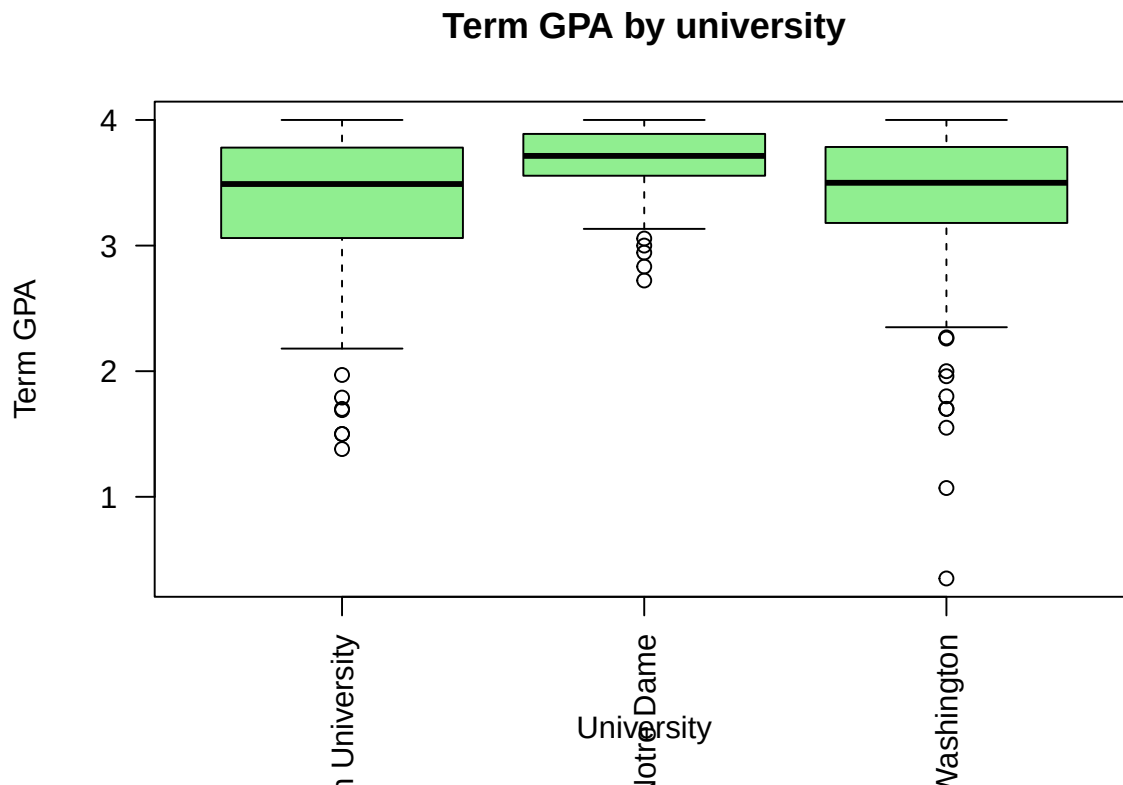
If instead of showing the frequency we want to show the density we need to indicate that the argument `freq=FALSE`. This means the height of each bar now represents how densely packed the data points are in that interval, rather than the raw count of observations. The total area under all bars equals 1 (like a probability).

```
hist(students$avg_sleep_hours,  
     main = "Average sleep hours",  
     xlab = "Hours",  
     col  = "lightblue",  
     freq = F)
```



A boxplot with the formula interface, `numeric ~ group`, compares a numeric variable across the levels of a categorical one. The box itself represents the interquartile range (IQR), with the bottom edge at the first quartile (25th percentile) and the top edge at the third quartile (75th percentile). The median (50th percentile) is indicated with a horizontal line inside the box. The whiskers extend from the box to the furthest data point within 1.5 times the IQR, and any observations beyond this range are plotted as individual points, which can be considered outliers. Read it as “term GPA by university.”

```
boxplot(term_gpa ~ university, data = students,  
        main = "Term GPA by university",  
        xlab = "University",  
        ylab = "Term GPA",  
        col = "lightgreen",  
        las = 2)
```



2.5 Counts across categories: barplot()

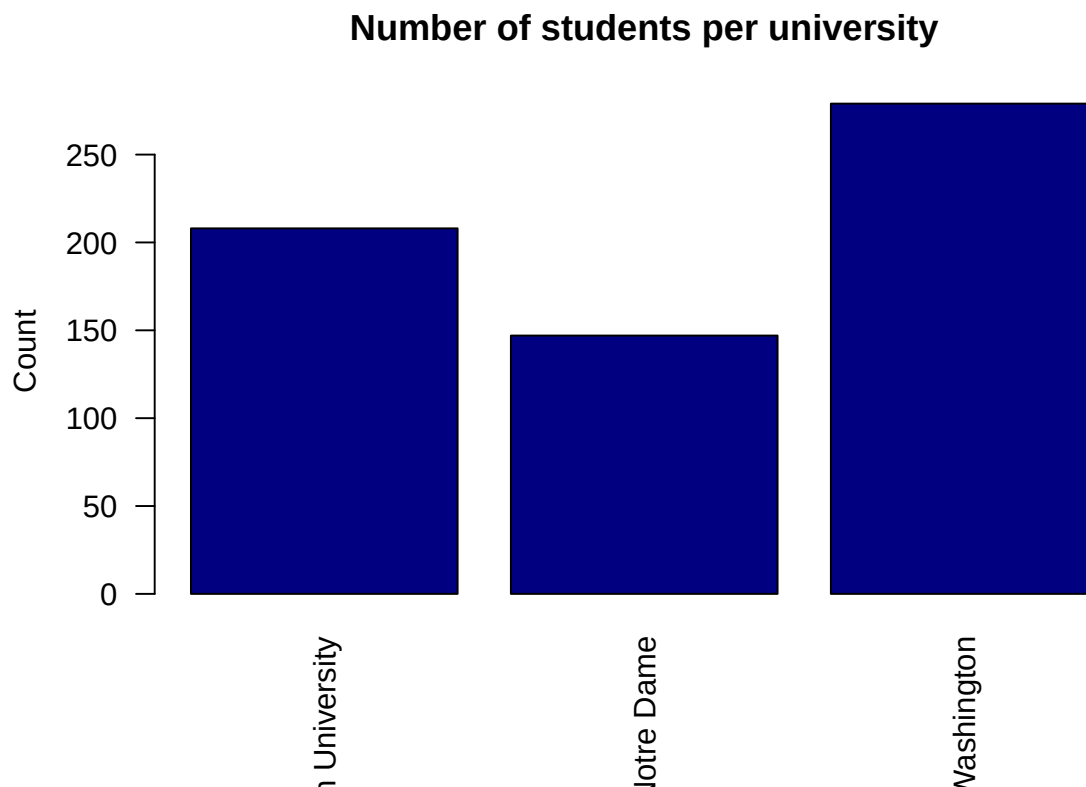
A barplot is a graphical representation used to display categorical data using rectangular bars. The height (or length) of each bar is proportional to the value it represents. Unlike histograms which show the distribution of continuous data, barplots show comparisons between different categories or groups.

When what you want is one number per category, how many, or an average, the right tool is a barplot. Each bar's height is that single value. The simplest case is a frequency count. `table()` counts how many rows fall in each level of a categorical variable, and `barplot()` draws those counts.

```
# How many students in each university?
counts <- table(students$university)
counts

##
## Carnegie Mellon University    University of Notre Dame
##                208                147
## University of Washington
##                279

barplot(counts,
        main = "Number of students per university",
        ylab = "Count",
        col = "navy",
        las = 2)
```



The `las = 2` turns the axis labels sideways so the long university names stay readable.

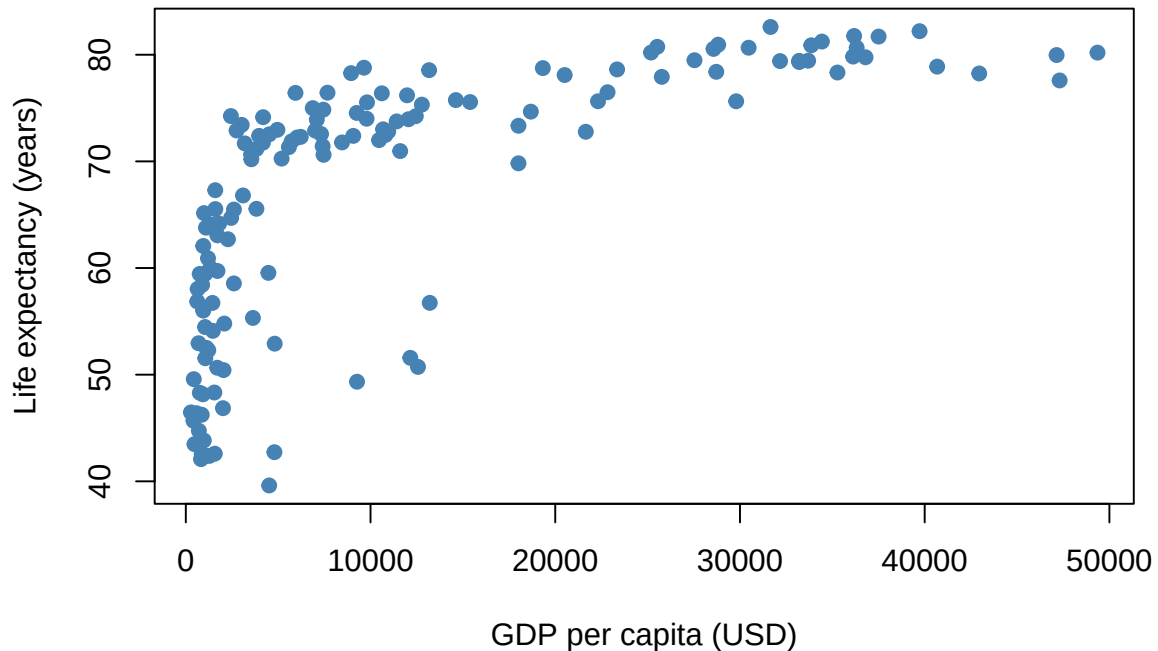
3 The log lesson: when an axis spans orders of magnitude

Now the case that bounded variables never show us. In `gapminder`, income per capita and life expectancy have a very particular shape. We take a single year so each country appears once.

```
gap07 <- gapminder |> filter(year == 2007)

plot(gap07$gdpPercap, gap07$lifeExp,
     main = "Life expectancy vs income per capita, 2007",
     xlab = "GDP per capita (USD)",
     ylab = "Life expectancy (years)",
     pch = 19, col = "steelblue")
```

Life expectancy vs income per capita, 2007



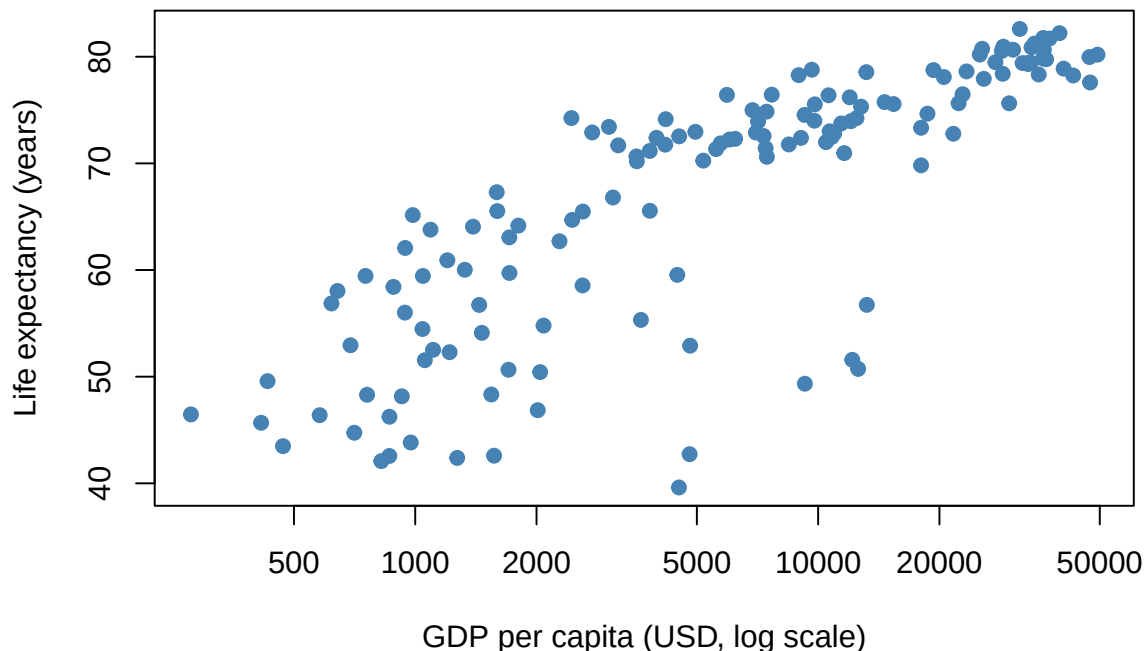
The cloud rises steeply among poorer countries then flattens off, a curve, not a line. The reason is that `gdpPercap` spans roughly three orders of magnitude (a few hundred dollars to over a hundred thousand), so most countries are crushed against the left edge. Let us measure how straight the relationship is, then put the income axis on a log scale with `log = "x"` and measure again.

```
# Correlation on the raw scale  
cor(gap07$gdpPercap, gap07$lifeExp)
```

```
## [1] 0.6786624
```

```
# Same plot, income axis on a log scale  
plot(gap07$gdpPercap, gap07$lifeExp,  
      log = "x",  
      main = "Life expectancy vs log income, 2007",  
      xlab = "GDP per capita (USD, log scale)",  
      ylab = "Life expectancy (years)",  
      pch = 19, col = "steelblue")
```

Life expectancy vs log income, 2007



```
# Correlation after logging income  
cor(log(gap07$gdpPercap), gap07$lifeExp)
```

```
## [1] 0.8089803
```

The curve straightens into a clear diagonal band, and the correlation rises noticeably. That is the whole lesson: when a predictor is heavily right-skewed, logging it both straightens the relationship and strengthens the correlation.

4 The tidyverse plots: ggplot2

Base R gave us readable exploratory plots quickly. But every extra piece of information meant another argument, and the moment we want to colour points by group, size them by a third variable, and log an axis all at once, base R fights us. That is the problem **ggplot2** was built to solve, and it is where Part B begins.

We will now see how **ggplot2** can produce more elegant graphs. **ggplot2** is a powerful R package (part of the **tidyverse**) which is designed especially for data visualisation and providing best exploratory data analysis. It provides beautiful, hassle-free plots that take care of minute details like drawing legends and representing them. The plots can be created iteratively and edited later. This package is designed to work in a layered fashion, starting with a layer showing the raw data collected during exploratory data analysis with R then adding layers of annotations and statistical summaries.

Every ggplot has three core pieces:

- The data (data).

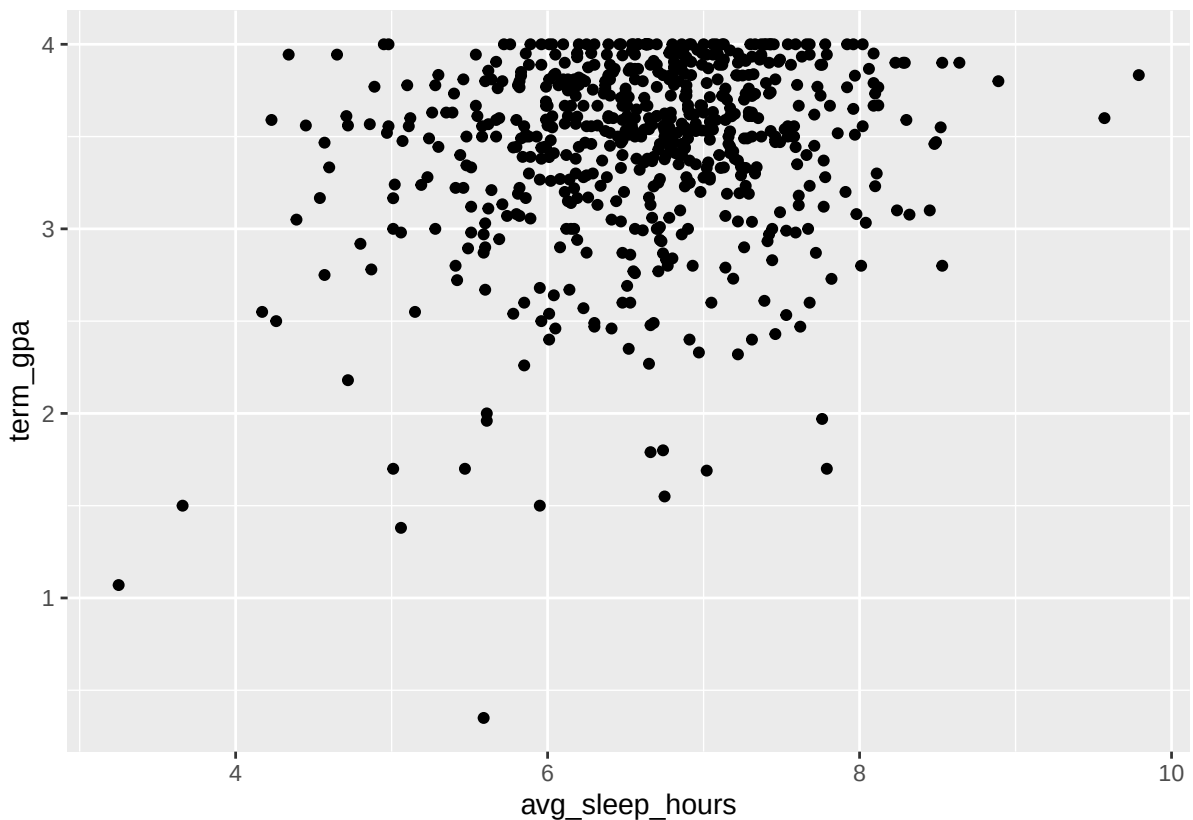
- How data points are mapped to each other (aes).
- A set of geometric objects also referred to as geoms (`geom_`), visual marks that represent the data points like e.g. bar, dots, or lines).

In addition to the base layers, `ggplot2()` offers many other layers to adjust plots through colouring, scaling, themes, and faceting or adding statistical transformations to the data.

4.1 `geom_point()`

We build the sleep-and-GPA scatter from section 2.1 the `ggplot` way. Read it as “take the students data, map sleep to x and GPA to y, and draw points.”

```
ggplot( students, aes(x = avg_sleep_hours, y = term_gpa)) +  
geom_point()
```

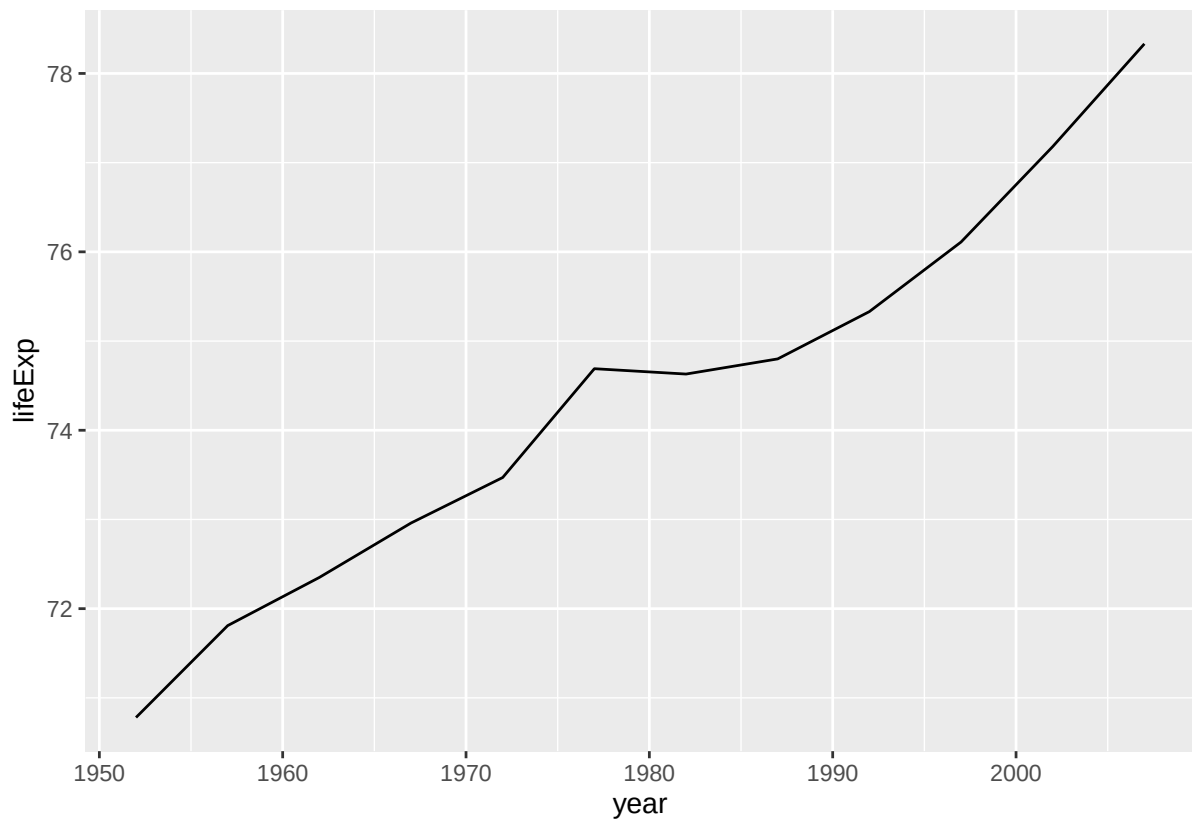


Notice the structure: `ggplot()` sets up the data and mapping, and `geom_point()` adds a layer. The `+` joins layers, and it always goes at the end of a line, never the start.

4.2 `geom_line()`

As in base R, we can also create a plot with lines instead.

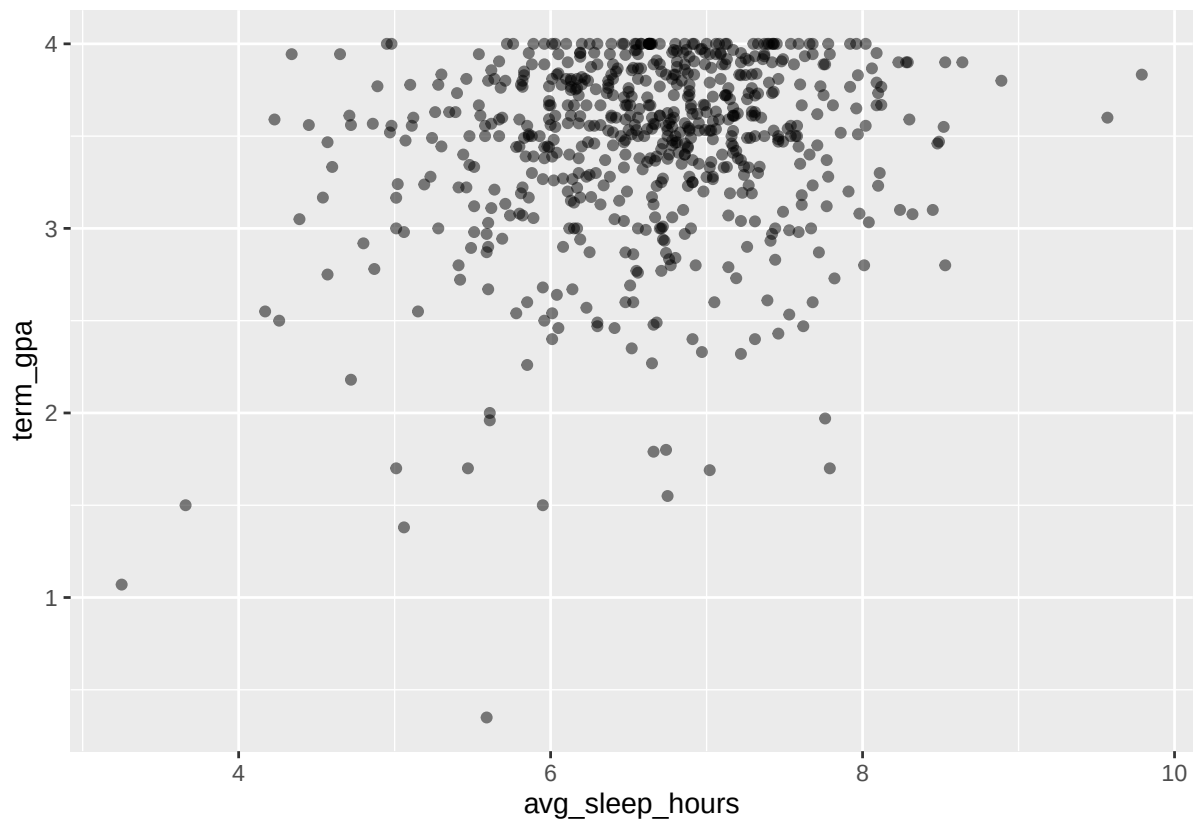
```
denmark |>  
ggplot(aes(x=year, y=lifeExp))+  
geom_line()
```



4.3 Transparency

In Part A we hit two problems with this scatter, a dense unreadable cluster (overplotting) and GPA flattening against its ceiling of 4. `ggplot` handles both cleanly. Transparency is set with `alpha` (0 is invisible, 1 is solid), placed inside the `geom` because it is a fixed setting, not a mapping from data.

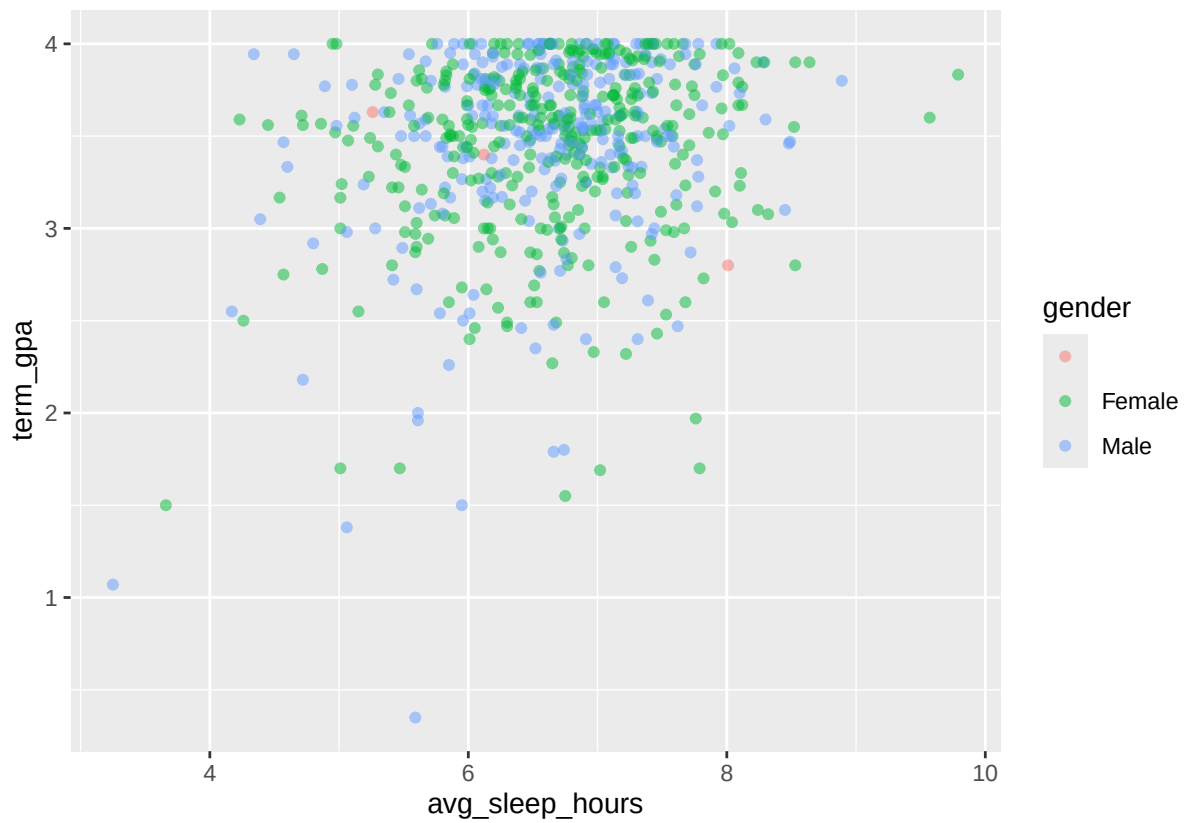
```
ggplot(students, aes(x = avg_sleep_hours, y = term_gpa)) +  
  geom_point(alpha = 0.5)
```



4.4 A third variable

Here is where ggplot pulls ahead of base R. To bring in a third variable, you map it to a new aesthetic, and ggplot handles the colours and the legend for you. The key idea: a variable goes inside `aes()`, a fixed setting goes outside it.

```
ggplot(students, aes(x = avg_sleep_hours, y = term_gpa, colour = gender)) +  
  geom_point(alpha = 0.5)
```

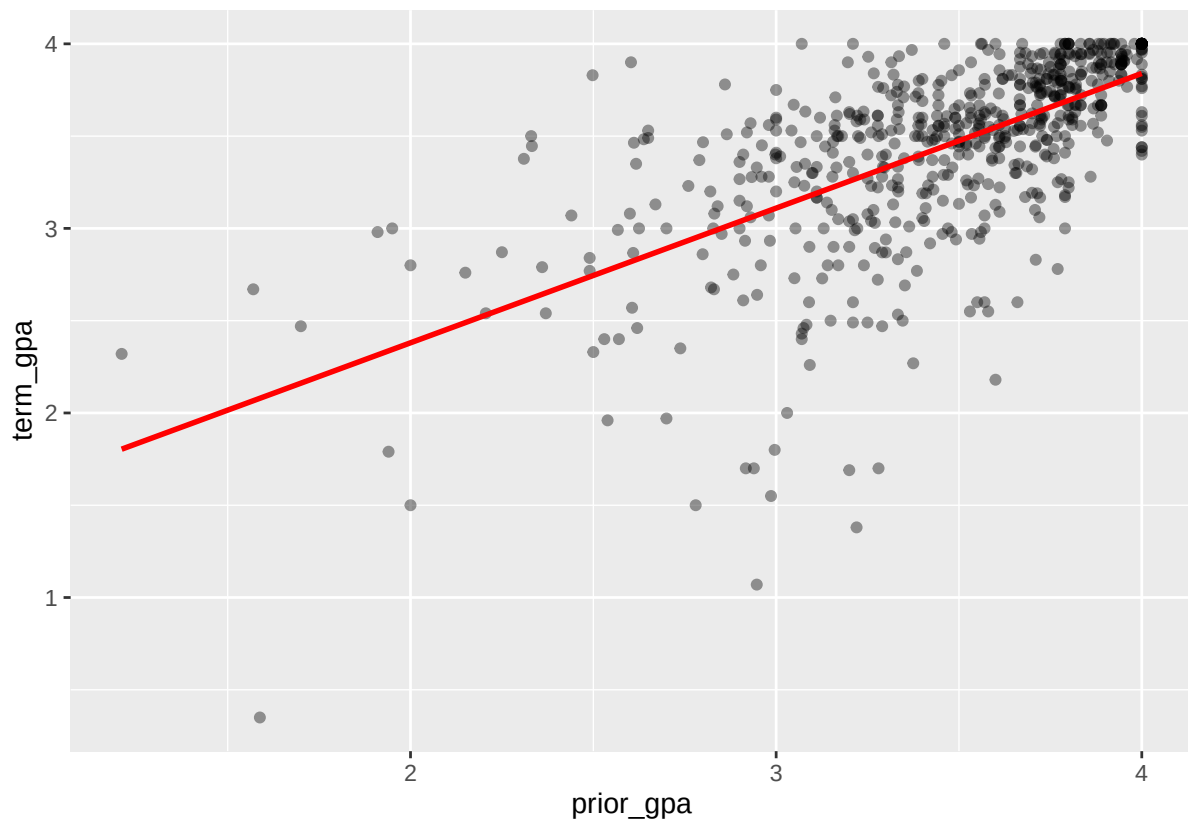


4.5 Adding a trend line: `geom_smooth()`

To show the overall direction through a cloud of points, add a `geom_smooth()` layer.

```
ggplot(students, aes(x = prior_gpa, y = term_gpa)) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm", se = FALSE, colour = "red")
```

'geom_smooth()' using formula = 'y ~ x'

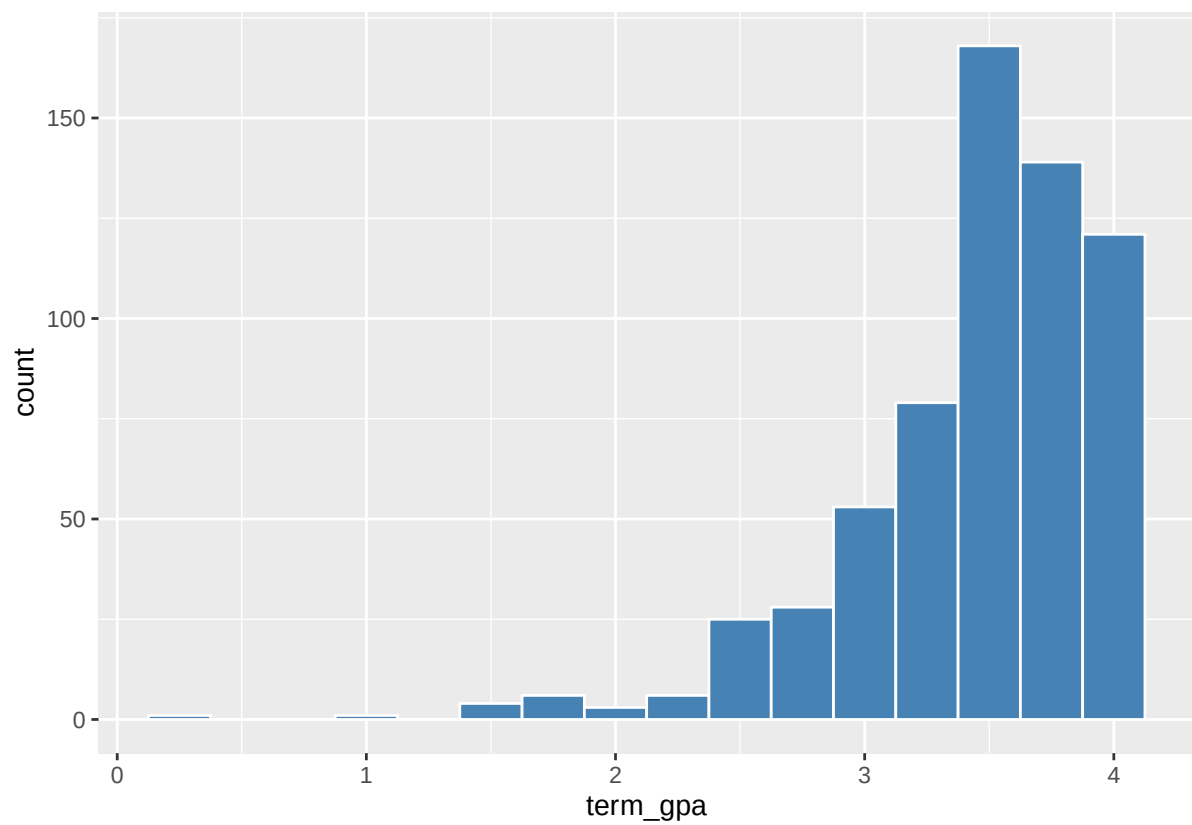


`ggplot` fits and draws the line for you here. What that line actually is, and how to read its slope and intercept, is the whole of the regression day, so treat this for now as the visual counterpart to the `abline()` you met in Part 2.3.

4.6 Distributions in `ggplot`: histogram and boxplot

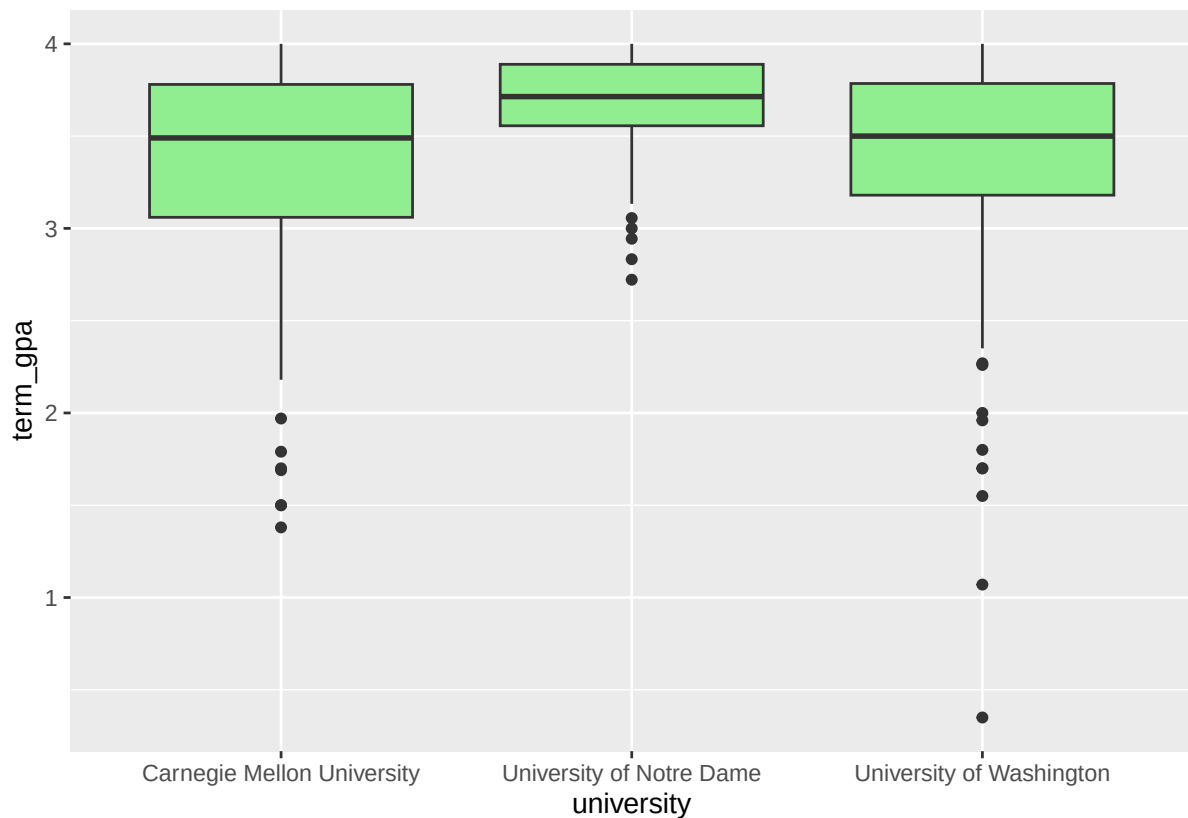
The distribution tools from Part A have direct `ggplot` equivalents. A histogram needs only an `x` mapping, since `ggplot` computes the counts.

```
ggplot(students, aes(x = term_gpa)) +  
  geom_histogram(binwidth = 0.25, fill = "steelblue", colour = "white")
```



A boxplot across groups maps the category to x and the numeric to y.

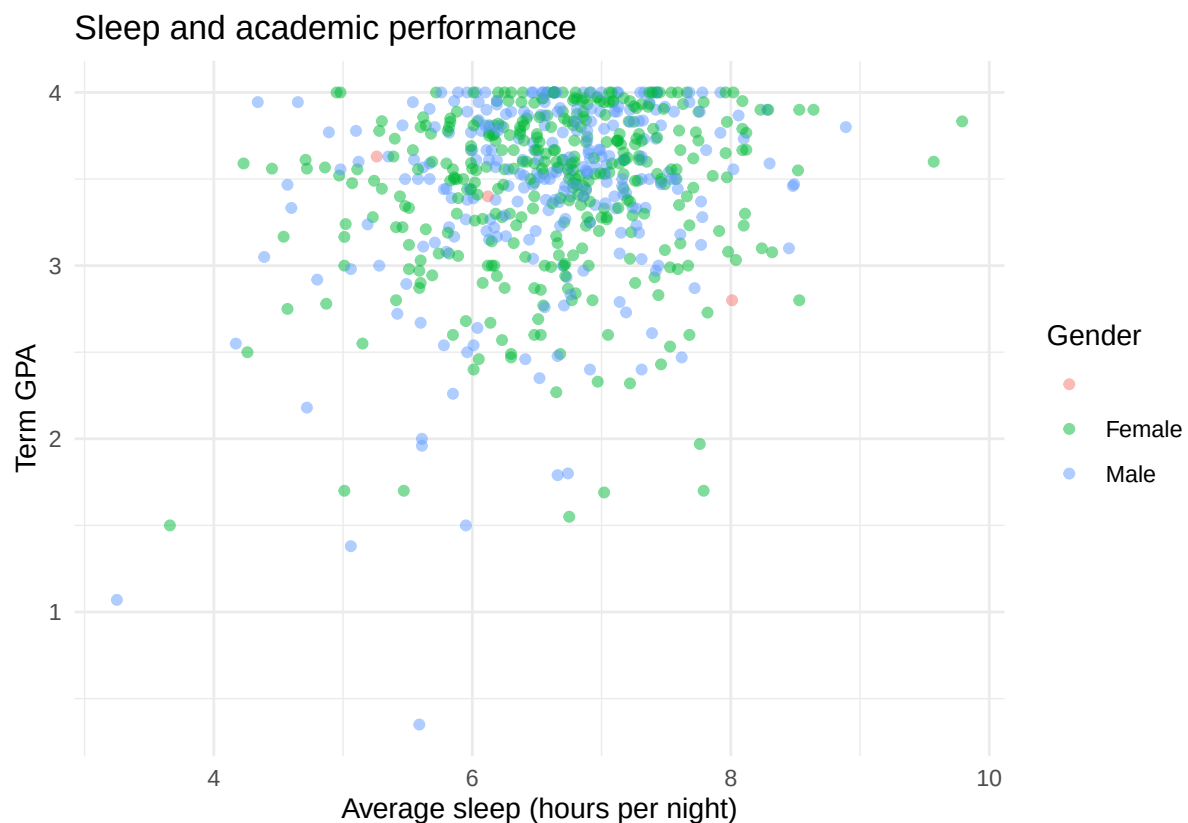
```
ggplot(students, aes(x = university, y = term_gpa)) +  
  geom_boxplot(fill = "lightgreen")
```



4.7 Labels, titles, and themes

A presentation plot needs readable labels and a title. `labs()` sets them all in one place, including the legend titles, and a `theme_*()` function restyles the whole plot.

```
ggplot(students, aes(x = avg_sleep_hours, y = term_gpa, colour = gender)) +
  geom_point(alpha = 0.5) +
  labs(title = "Sleep and academic performance",
       x = "Average sleep (hours per night)",
       y = "Term GPA",
       colour = "Gender") +
  # try different themes
  theme_minimal()
```

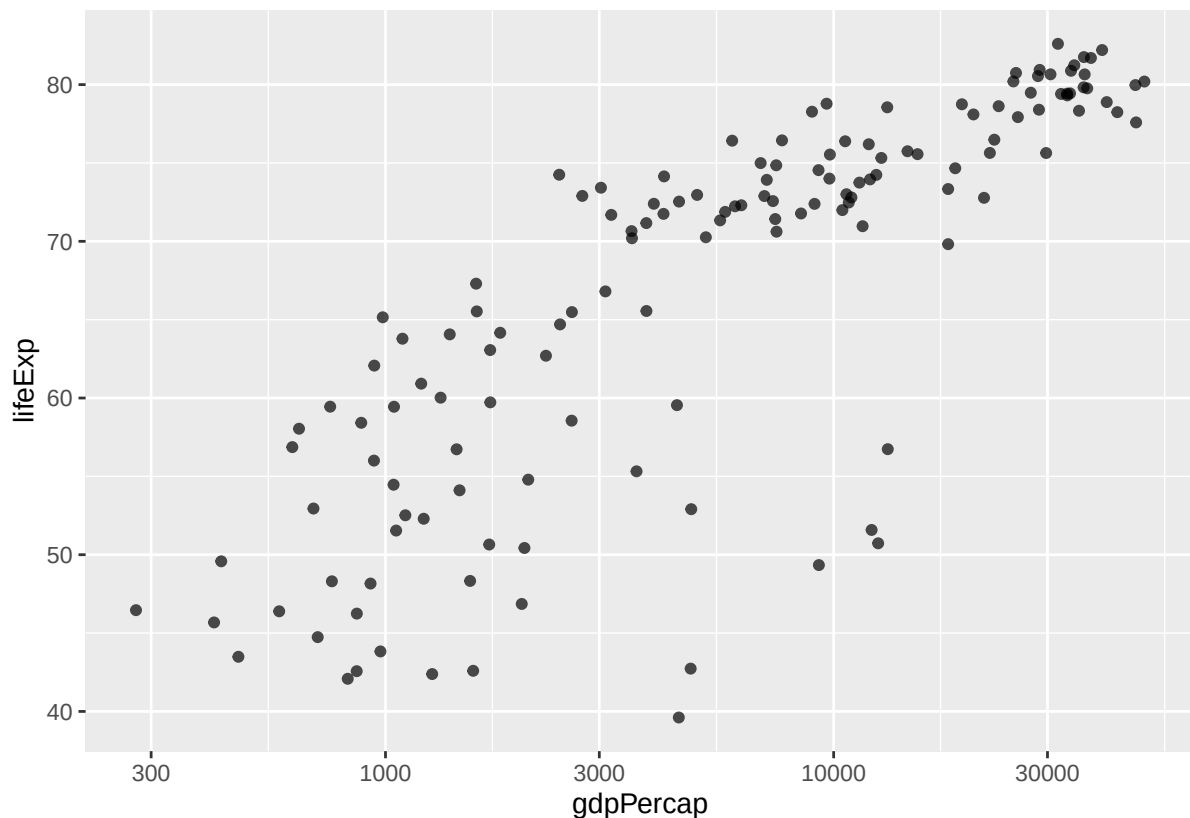


The rule of thumb: map your variables first, then reach for `labs()` and a theme last to make it presentable.

4.8 Scales and the log axis

Back to the gapminder income curve from Part A. In base R we logged the axis with `log = "x"`. In ggplot the same job is done by a scale layer, `scale_x_continuous(trans = "log10")`, which logs the axis while keeping the original dollar values on the labels.

```
ggplot(gap07, aes(x = gdpPercap, y = lifeExp)) +
  geom_point(alpha = 0.7) +
  scale_x_continuous(trans = "log10")
```

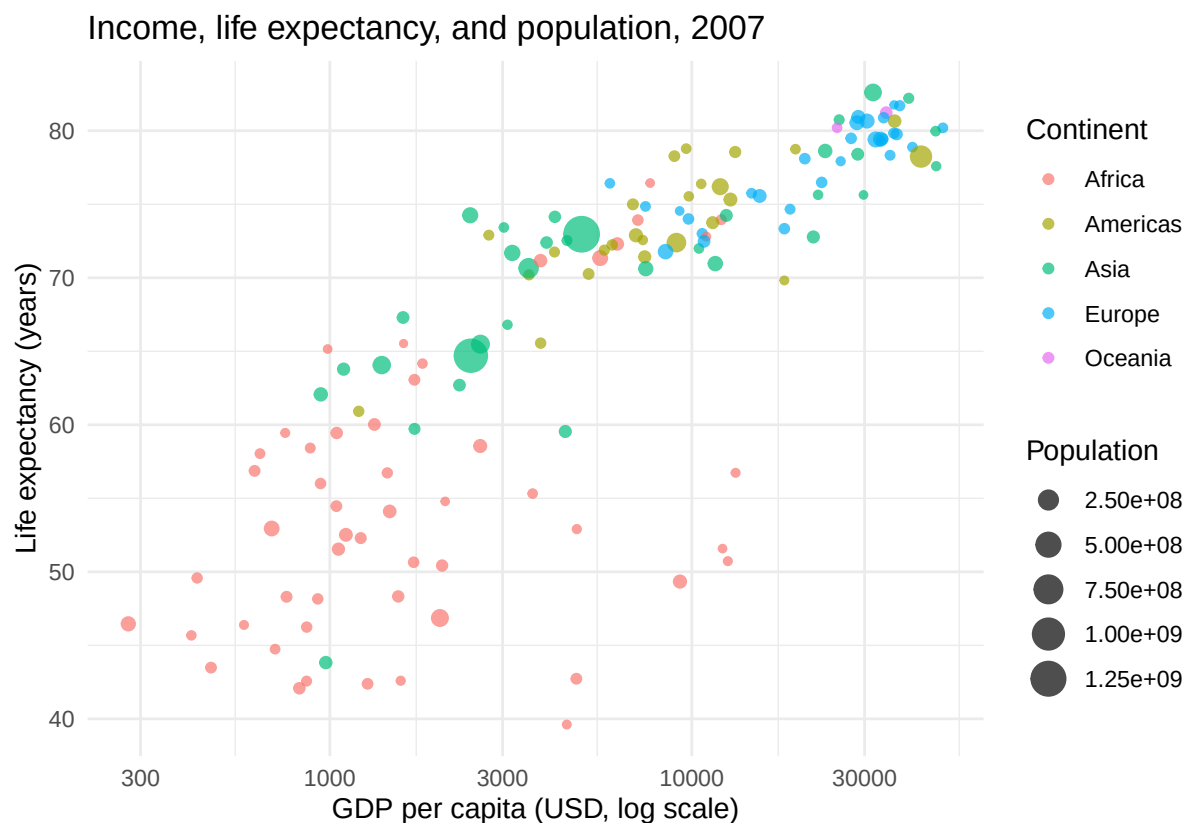


The curve straightens, exactly as it did in base R, and the axis still reads in dollars rather than log units.

4.9 Putting it all together: several variables in one plot

Now we combine everything, a scatter with a logged x axis, colour mapped to one variable, and size mapped to another, all readable at once. This is the plot that shows why ggplot exists, and it is structurally the same plot the exam asks you to build.

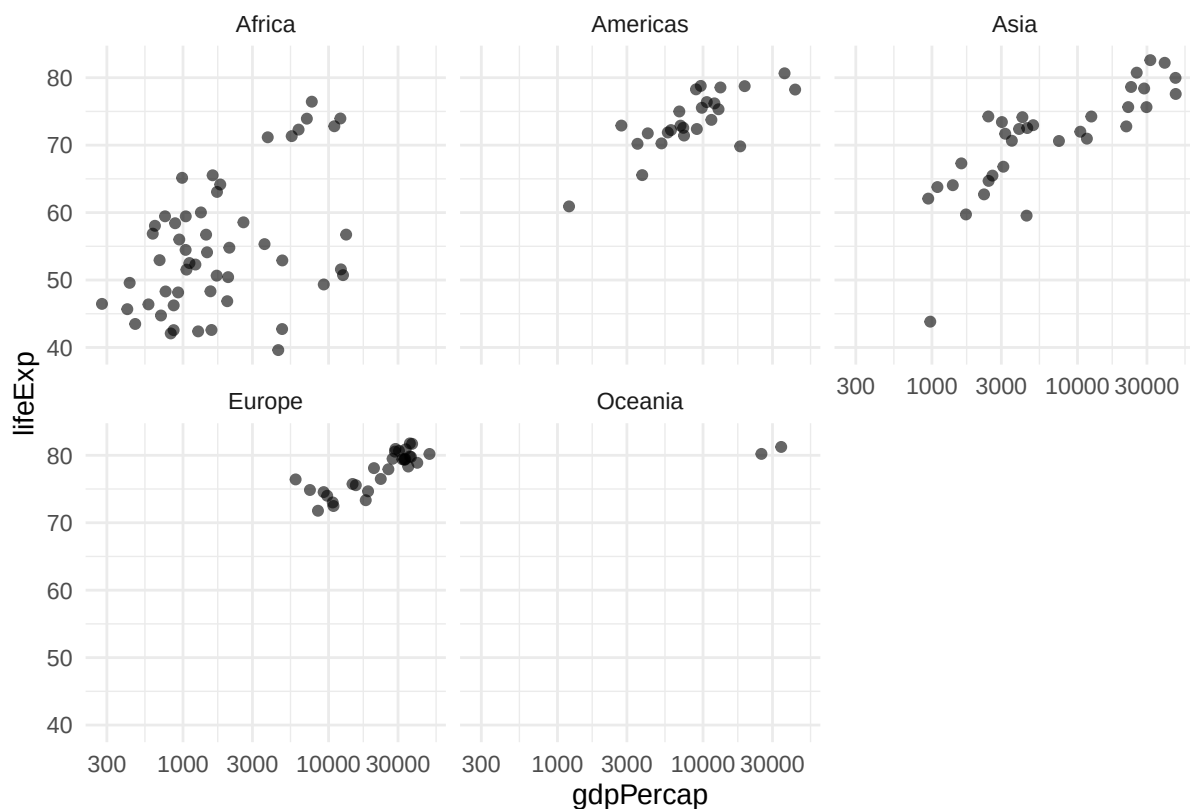
```
ggplot(gap07, aes(x = gdpPercap, y = lifeExp,
                  colour = continent, size = pop)) +
  geom_point(alpha = 0.7) +
  scale_x_continuous(trans = "log10") +
  labs(title = "Income, life expectancy, and population, 2007",
       x = "GDP per capita (USD, log scale)",
       y = "Life expectancy (years)",
       colour = "Continent",
       size = "Population") +
  theme_minimal()
```



Read what this single plot tells you: income rises left to right on a log scale, life expectancy climbs with it, each continent has its own colour, and the size of each point shows the country's population. Four variables, one readable picture.

4.10 Multiple panels with `facet_wrap()`

```
ggplot(gap07, aes(x = gdpPercap, y = lifeExp)) +
  geom_point(alpha = 0.6) +
  scale_x_continuous(trans = "log10") +
  facet_wrap(~ continent) +
  theme_minimal()
```



Each continent now gets its own panel on shared axes, which makes within-group patterns easy to compare.

5 Missing data: package `naniar`

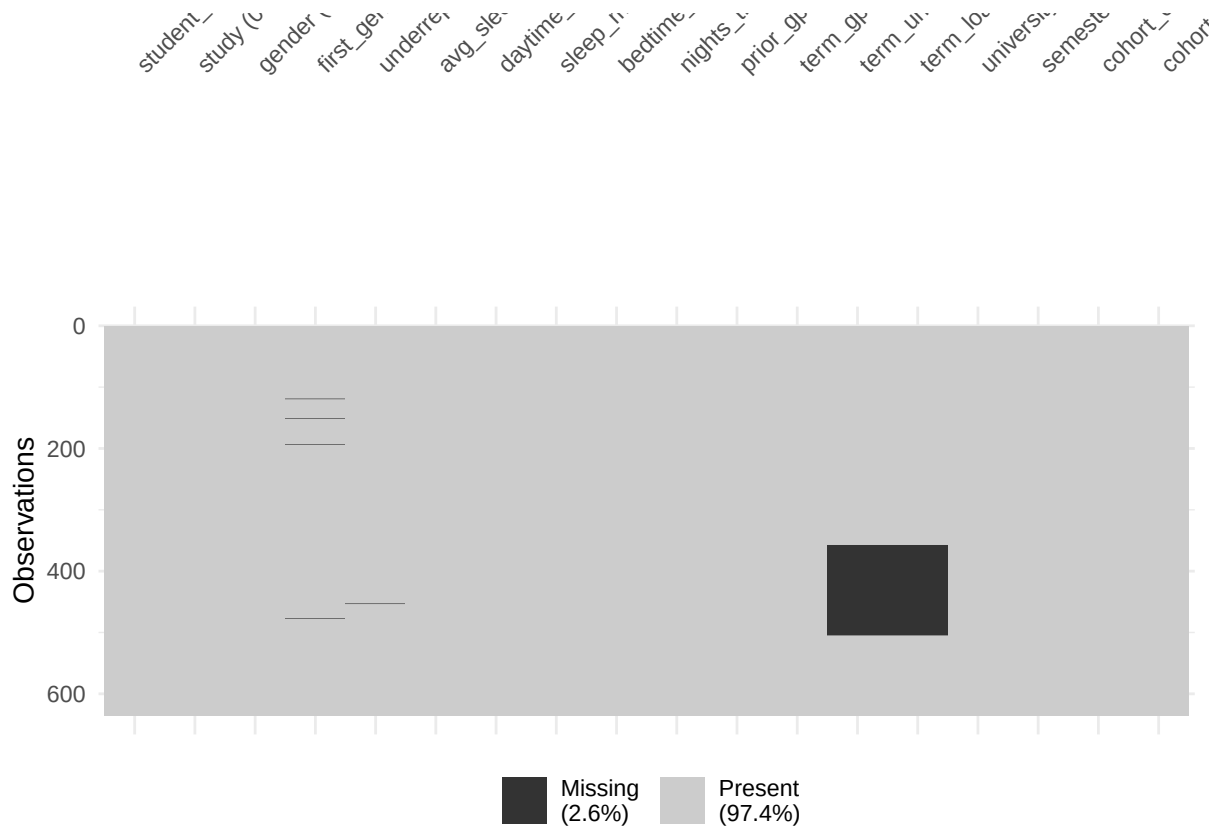
Across the course we have found missing values with `is.na()` and `colSums()`, and decided what to do with them. What we have not done is look at them. The `naniar` package draws missingness so you can see its pattern at a glance, and pattern is exactly what a column of counts hides.

You may have already spotted a preview: in the colour-by-gender scatter earlier, a faint unlabelled colour sat above Female and Male in the legend. Those were the handful of students with no recorded gender, missing values showing up uninvited in a plot. `naniar` lets us look at them on purpose.

```
# install.packages("naniar")
library(naniar)
```

The first tool, `vis_miss()`, draws the whole dataset as a grid, one row per student, one column per variable, with missing cells marked. It is the fastest way to see whether missingness is scattered at random or concentrated.

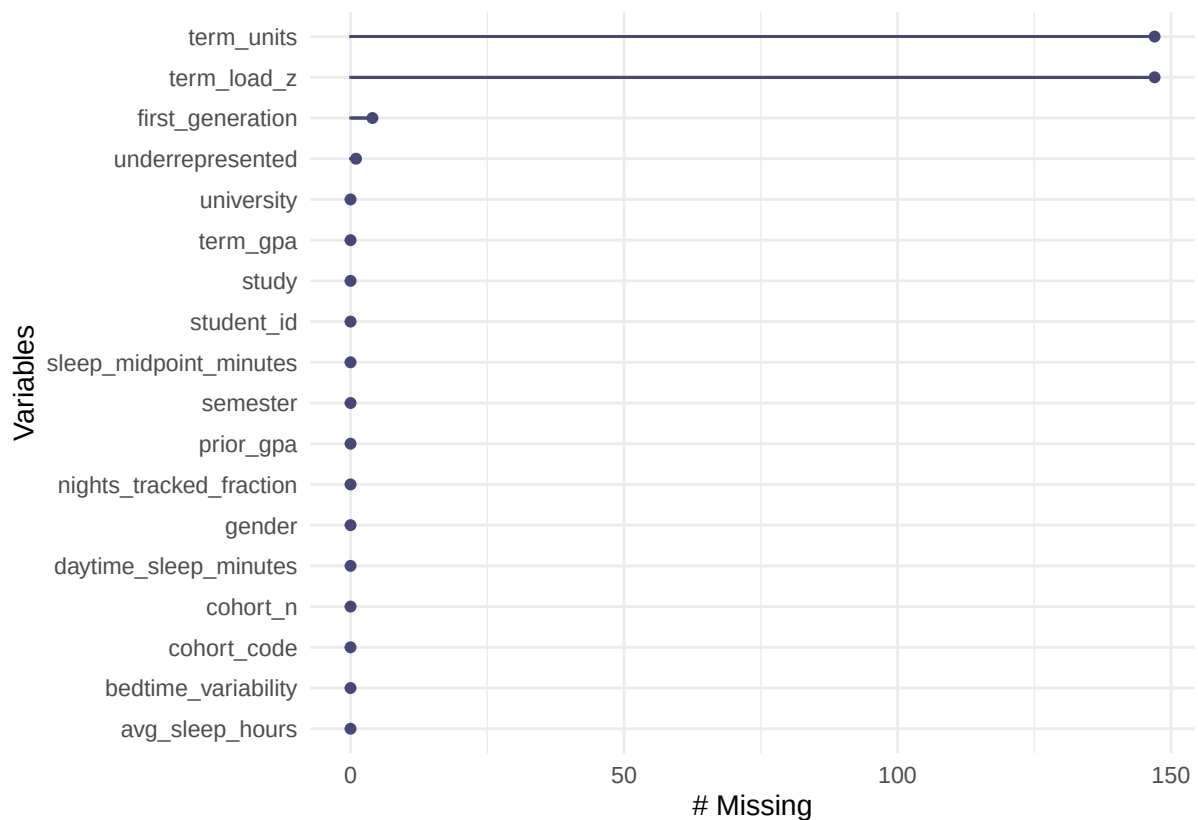
```
vis_miss(students)
```



Two things jump out. The scattered gaps in `gender`, `first_generation`, and `underrepresented` are a few students each. But `term_units` and `term_load_index` are missing in a solid block: every student from one university (Notre Dame) has no units recorded, because that study simply did not collect it. That is a completely different kind of missingness from a few stray blanks, and you would never see the difference from a count alone.

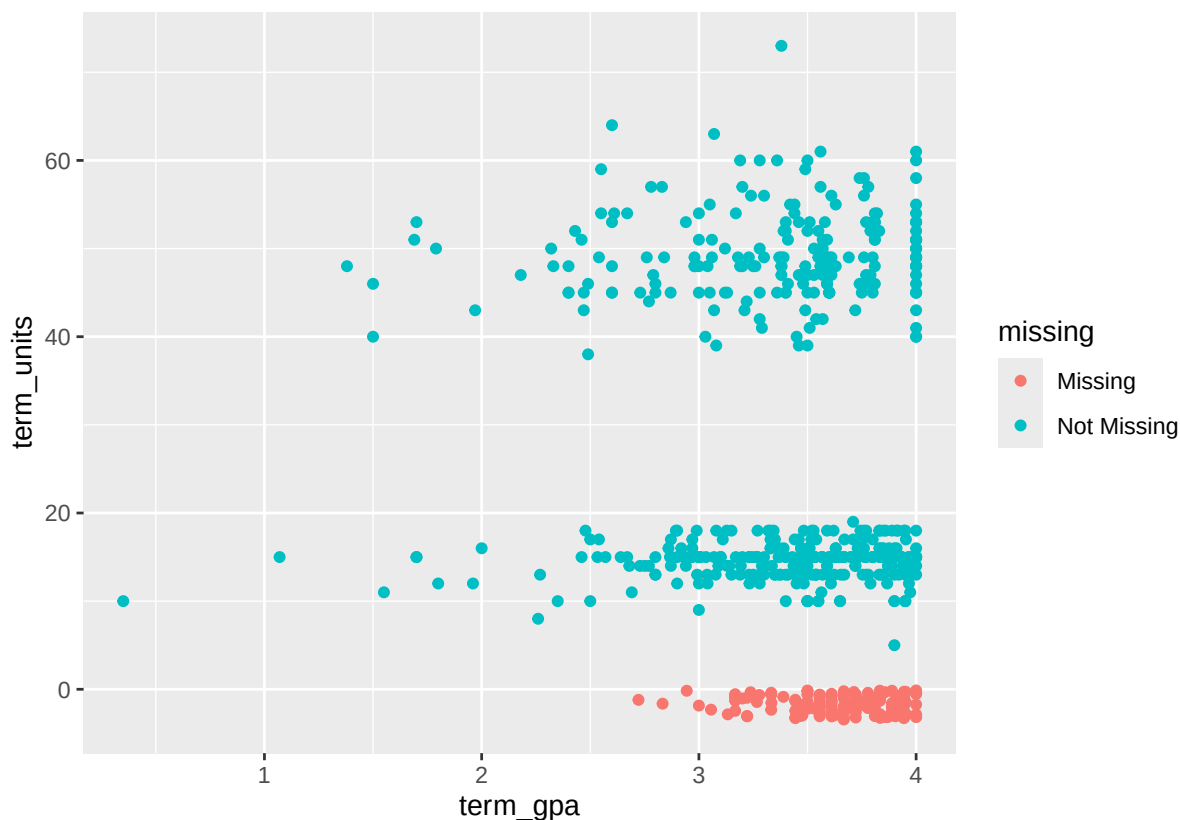
The second tool, `gg_miss_var()`, ranks the variables by how many values each is missing. It answers “which columns should I worry about?” directly.

```
gg_miss_var(students)
```



The third tool solves a quieter problem. When you scatter two variables, ggplot silently drops any point missing on either axis, so the plot looks complete while hiding the gaps. `geom_miss_point()` instead places those dropped points in the margin, in a separate colour, so you can see how many there were and whether they differ.

```
ggplot(students, aes(x = term_gpa, y = term_units)) +  
  geom_miss_point()
```



One caution to carry forward: naniar only sees genuine NA. If a dataset stores missing values as a stand-in like 0 or -99 (as we saw earlier in the course), naniar will treat those as real numbers until you convert them to NA first. Seeing missingness depends on it actually being marked as missing.

6 Combining plots: patchwork()

When you want two or three plots side by side as one figure (for a report, or an exam answer), the patchwork package lets you combine saved ggplot objects with simple arithmetic. You store each plot in a variable, then lay them out.

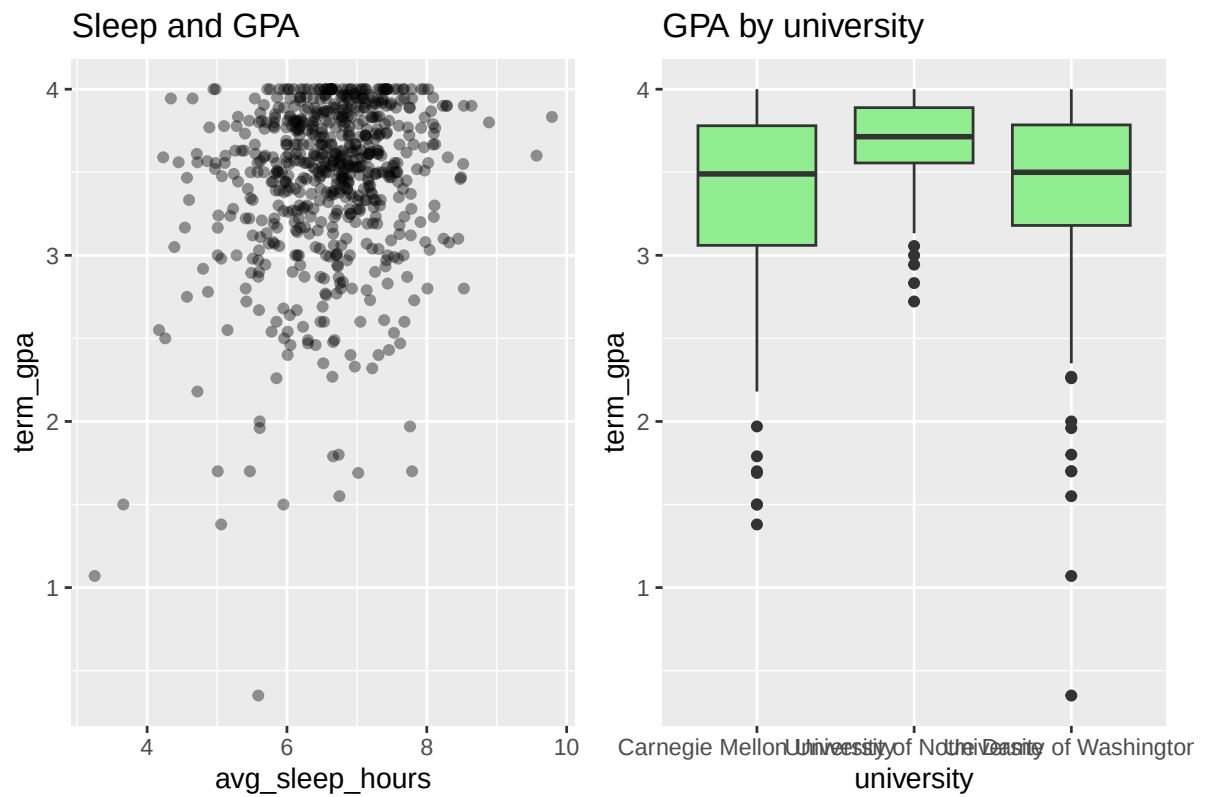
```
# install.packages("patchwork")
library(patchwork)

p1 <- ggplot(students, aes(x = avg_sleep_hours, y = term_gpa)) +
  geom_point(alpha = 0.4) +
  labs(title = "Sleep and GPA")

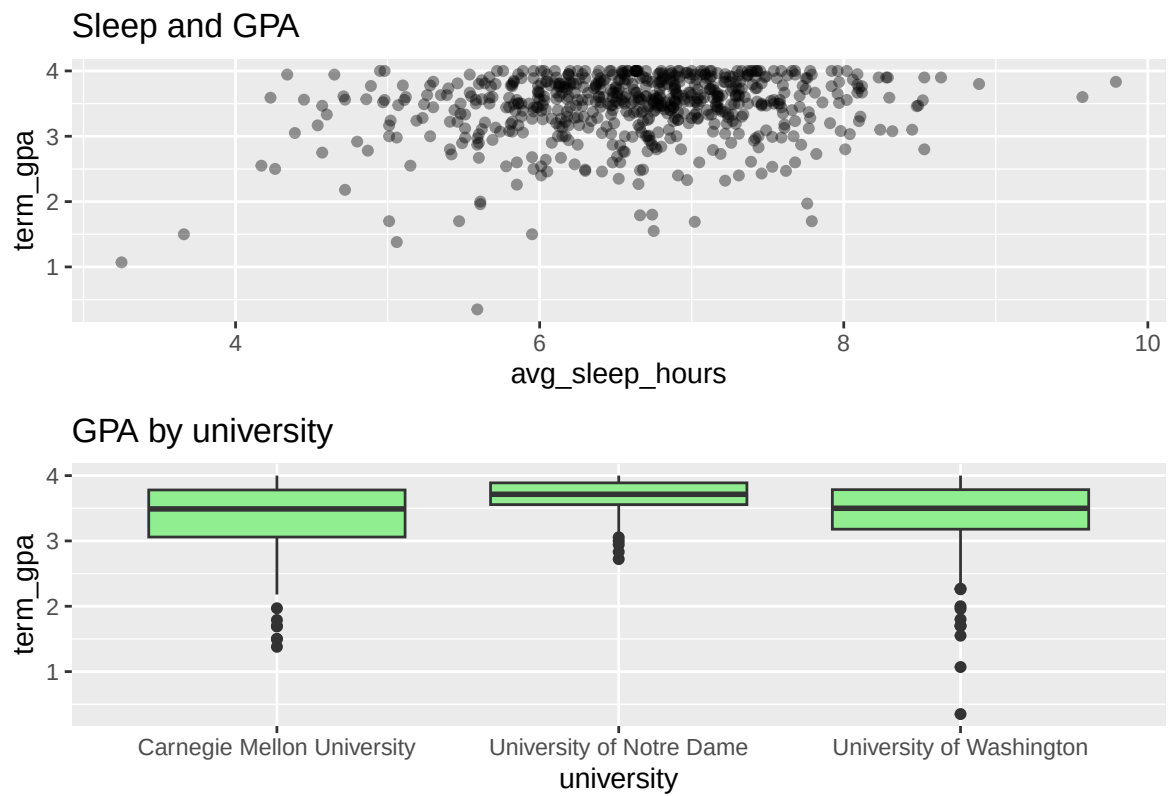
p2 <- ggplot(students, aes(x = university, y = term_gpa)) +
  geom_boxplot(fill = "lightgreen") +
  labs(title = "GPA by university")
```

A + places plots side by side, and a / stacks them.

p1 + p2 # side by side



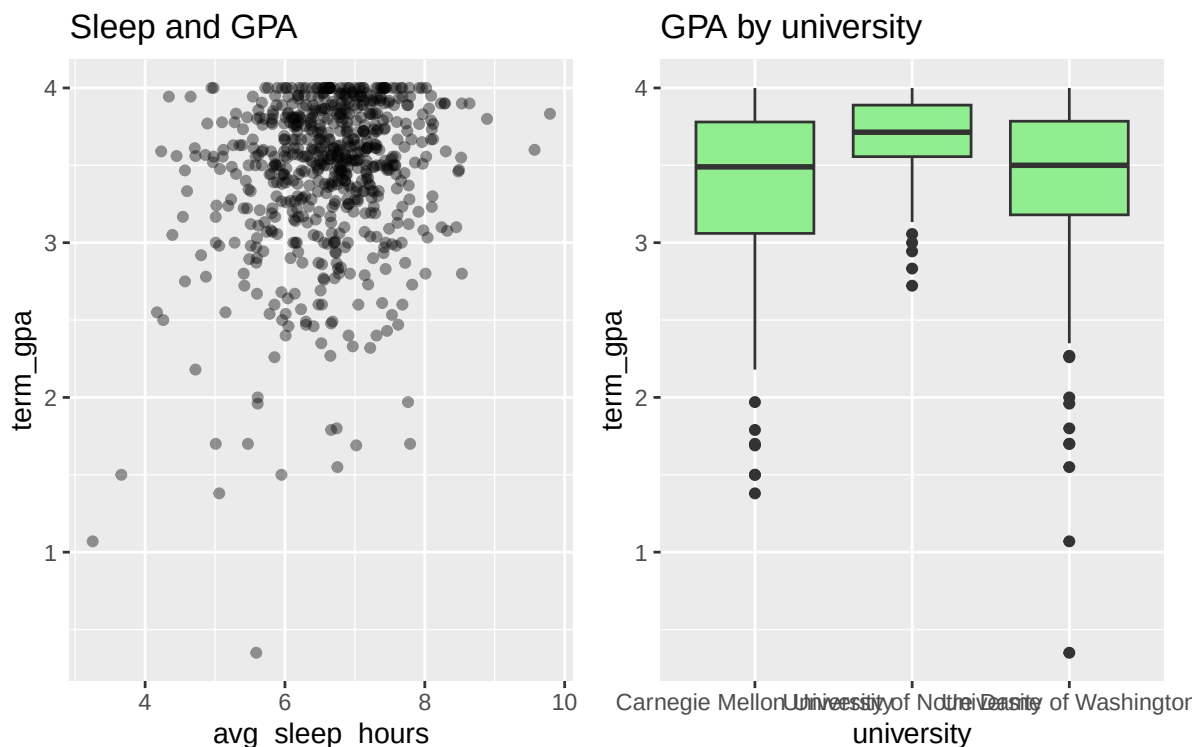
p1 / p2 # stacked



You can group with parentheses to build a layout, and `plot_annotation()` adds one shared title over the whole figure.

```
(p1 + p2) +  
  plot_annotation(title = "Two views of term GPA")
```

Two views of term GPA



The rule to remember: build and polish each plot on its own, then use patchwork only to arrange the finished pieces.

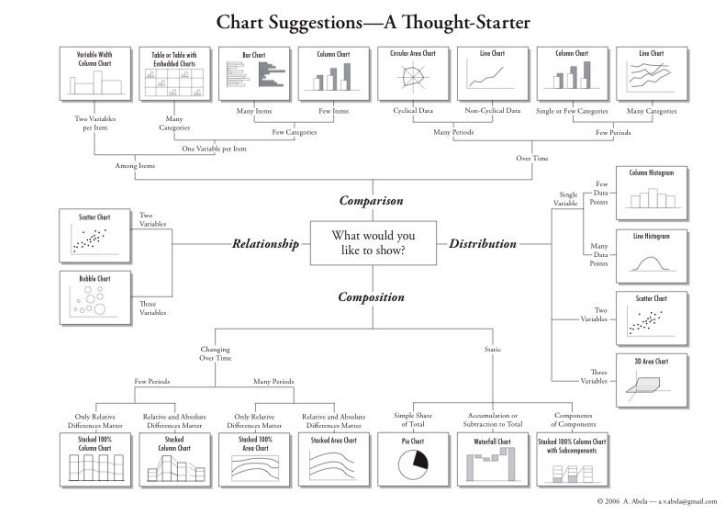
7 What viz to choose?

Most of the work in a plot is choosing the right type for what you want to show. This table covers the five you have learned today.

Data type	Purpose	Best plot	R function
One numeric variable	Show its distribution and spread	Histogram	<code>hist()</code>
One numeric variable across groups	Compare a distribution between categories	Boxplot	<code>boxplot(y ~ group)</code>
One categorical variable	Count or compare categories	Barplot	<code>barplot(table(x))</code>
Two numeric variables	Show a relationship	Scatter plot	<code>plot(x, y)</code>
A numeric variable over time	Show a trend	Line plot	<code>plot(x, y, type = "l")</code>

In ggplot2 the same five are `geom_histogram()`, `geom_boxplot()`, `geom_point()` (scatter), and `geom_line()` (trend).

For a wider map of chart types beyond this course, the flowchart below is a useful reference to keep.



8 Glossary

The functions used today, grouped by what they are for.

Base R plotting

Function	What it does
1. <code>plot()</code>	Draws a scatter of x and y, or a line with <code>type = "l"</code>
2. <code>abline()</code>	Adds a straight line to an existing plot (<code>h =</code> , <code>v =</code> , or <code>a =/b =</code>)
3. <code>hist()</code>	Histogram showing the distribution of one numeric variable
4. <code>boxplot()</code>	Distribution of a numeric across groups, using <code>numeric ~ group</code>
5. <code>barplot()</code>	One value per category (a count or a summary)
6. <code>table()</code>	Counts how many rows fall in each level of a categorical variable

ggplot2

Function	What it does
7. <code>ggplot()</code>	Starts a plot, setting the data and the aesthetic mapping
8. <code>aes()</code>	Maps variables to visual channels (x, y, colour, size)
9. <code>geom_point()</code>	Draws the data as points (a scatter)
10. <code>geom_line()</code>	Draws the data as a line
11. <code>geom_smooth()</code>	Adds a trend line, straight with <code>method = "lm"</code>

Function	What it does
12. <code>geom_histogram()</code>	Histogram of one numeric variable
13. <code>geom_boxplot()</code>	Boxplot of a numeric across groups
14. <code>labs()</code>	Sets the title, axis labels, and legend titles
15. <code>theme_minimal()</code>	Restyles the whole plot (one of the theme_ family)
16. <code>scale_x_continuous()</code>	Adjusts the x axis, including trans = "log10" for a log scale
17. <code>facet_wrap()</code>	Splits the plot into one panel per group, with ~ variable

Missing data (**naniar**)

Function	What it does
18. <code>vis_miss()</code>	Maps missing values across the whole dataset
19. <code>gg_miss_var()</code>	Ranks variables by how many values they are missing
20. <code>geom_miss_point()</code>	Scatter that shows, rather than drops, rows missing on one axis

Combining plots (**patchwork**)

Function	What it does
21. <code>+</code>	Places saved plots side by side
22. <code>/</code>	Stacks saved plots
23. <code>plot_annotation()</code>	Adds a shared title over a combined figure

9 Exercises

Work in a fresh script. For the gapminder questions, first filter to have data only for 1997.

- 1) Labelled scatter. Using `students`, plot `daytime_sleep_minutes` on x against `term_gpa` on y. Add a title, axis labels, a colour, and filled points (`pch = 19`). Then add a dashed horizontal reference line at the mean term GPA.
- 2) Distributions. (a) Draw a histogram of `avg_sleep_hours`. (b) Draw a boxplot of `term_gpa` by gender using the numeric `~ group` form. In one sentence, what does the comparison suggest?
- 3) Counts. Use `table()` to count how many students fall in each **gender**, then draw the result as a barplot with a title and a y axis label.
- 4) The log lesson. With `gap97`, scatter `gdpPercap` against `lifeExp`. Then draw the same plot with the income axis on a log scale. Compute the correlation before and after logging `gdpPercap` (`cor()`), and write one sentence explaining whether it changed and why.
- 5) Several variables in one plot. With `gap97`, build a single ggplot scatter of `gdpPercap` (log10 axis) against `lifeExp`, with colour mapped to continent and size mapped to pop. Add a title, axis labels, legend titles, and a theme. Write one sentence describing what the plot shows.